



Universidad Nacional Experimental de Guayana.
Vicerrectorado Académico.
Coordinación General de Pregrado.
Proyecto de Carrera: Ingeniería en Informática.
Unidad Curricular: Ingeniería de Software II.

Unidad III.

Ejercicio 2: La Falacia de los Recursos y el Cono de Incertidumbre.

Team Pixel & Code

“Estrategia, desarrollo y valor real”

Profesor:

Ing. Félix Márquez

Integrantes:

C.I.: V-28.530.717. Bravo Yvanna
C.I.: V-28.726.605. Guilarte Keibel
C.I.: V-19.505.397. Rivas Miguel
C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, 10 de Septiembre del 2026.

Contexto del problema

Faltan 2 meses para la fecha límite de un proyecto de software crítico. El proyecto está retrasado. El Director de la Empresa, ignorando la Ley de Brooks, ordena duplicar el equipo de desarrollo (de 5 a 10 programadores) para recuperar el tiempo perdido y cumplir con el cronograma de Gantt original.

Análisis: ¿por qué falla $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$?

La ecuación $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$ asume que el trabajo de software es perfectamente divisible y paralelizable, como si fuera trabajo físico repetitivo (por ejemplo, pintar una pared: si un pintor tarda 10 horas, dos pintores tardan 5). Esta suposición es una falacia en ingeniería de software por dos razones fundamentales:

1. El esfuerzo no es fungible ni divisible linealmente: Añadir personal no reduce el trabajo restante en la misma proporción, porque gran parte del "esfuerzo" no es escribir código nuevo, sino entender el código, la arquitectura y el contexto del proyecto ya existente. Ese conocimiento no se transfiere instantáneamente.

2. Ley de Brooks (Brooks, 1975): *"Agregar mano de obra a un proyecto de software retrasado lo retrasa más."* Esto ocurre porque el tiempo total no depende solo del esfuerzo, sino de la complejidad de coordinación, que crece de forma no lineal (aproximadamente cuadrática) con el número de personas, mientras que la capacidad productiva crece de forma mucho más lenta —y en las primeras semanas, incluso puede volverse negativa (el equipo produce menos que antes).

Matemáticamente, la ecuación real se parece más a:

$$\text{Tiempo} \approx \text{Esfuerzo} / (\text{Recursos} - \text{Sobrecarga}(\text{Recursos}))$$

Donde $\text{Sobrecarga}(\text{Recursos})$ crece más rápido que los propios recursos añadidos, llegando a un punto en que sumar gente resta capacidad neta en el corto plazo.

Esto también se conecta con el Cono de Incertidumbre (McConnell, 2004): al haber ya avanzado el proyecto, la estimación original debería tener menor varianza que al inicio; pero el Director está tratando la fecha original (fijada bajo alta incertidumbre inicial, $\pm 4\times$) como si fuera un compromiso rígido, ignorando que el cronograma de Gantt original probablemente ya estaba basado en una estimación optimista de baja fidelidad.

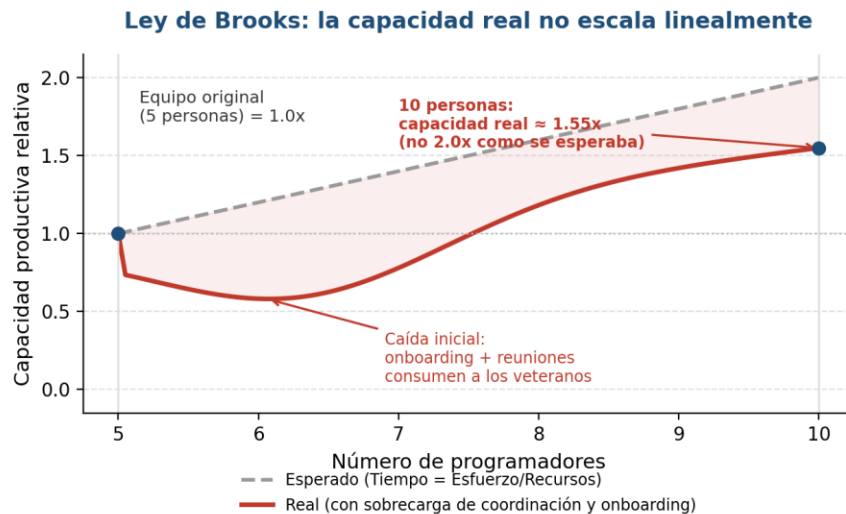


Figura 1. Al duplicar el equipo, la capacidad productiva real cae inicialmente (onboarding y comunicación) y solo se recupera parcialmente, quedando muy por debajo del 2.0x esperado.

a) Tres (3) factores de sobrecarga al duplicar el equipo

1. Comunicación (overhead de comunicación)

El número de canales de comunicación interpersonales dentro de un equipo se calcula mediante la fórmula combinatoria:

$$C = \frac{n(n - 1)}{2}$$

- **Con 5 desarrolladores:** $\frac{C=5(4)}{2} = 10$ canales de comunicación.
- **Con 10 desarrolladores:** $\frac{C=10(9)}{2} = 45$ canales de comunicación.

Duplicar el personal incrementa los canales de comunicación en un **350%** (de 10 a 45). El tiempo que antes se dedicaba a programar ahora se consume en reuniones de alineación, chats, correos y resolución de malentendidos, degradando la productividad neta del equipo.

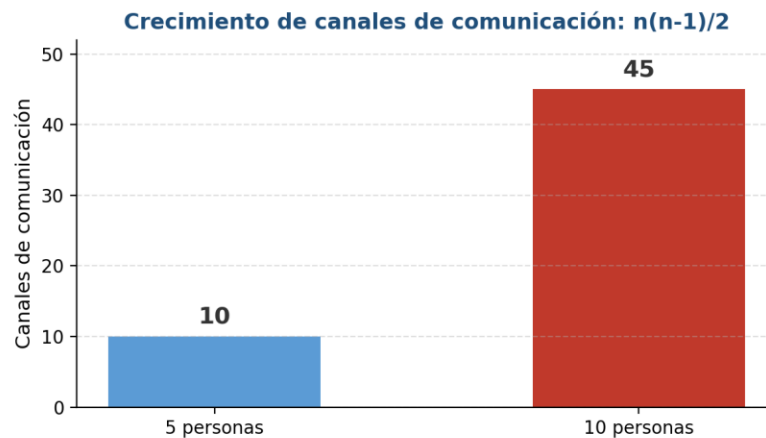


Figura 2. El número de canales de comunicación crece de forma cuadrática, no lineal, al duplicar el equipo.

2. Onboarding (curva de aprendizaje)

Los 5 desarrolladores nuevos no pueden ser productivos de inmediato. Necesitan tiempo para entender la arquitectura del sistema, las convenciones del código, las decisiones de diseño ya tomadas y el dominio del negocio. Durante este período consumen el tiempo de los desarrolladores originales (que deben explicarles y revisar su trabajo), reduciendo temporalmente la productividad del equipo veterano en lugar de aumentarla.

3. Integración (fricción técnica y de código)

Más desarrolladores trabajando en paralelo sobre el mismo código base incrementan la probabilidad de conflictos de merge, dependencias cruzadas mal coordinadas y regresiones. Se requiere más esfuerzo en integración, pruebas y resolución de conflictos, lo que aumenta el "código pegamento" y la deuda técnica generada bajo presión.

b) Dos (2) alternativas de gestión para la crisis de cronograma

Alternativa 1. Enfoque predictivo tradicional: reducir el alcance (Scope Cutting) manteniendo el cronograma

En un enfoque predictivo (PMBOK/Gantt), de las tres variables clásicas —alcance, tiempo, costo— si el tiempo es fijo (la fecha límite no puede moverse) y el costo ya se disparó, la única palanca viable es reducir el alcance. Esto implica:

- Re-priorizar el backlog usando MoSCoW (Must have / Should have / Could have / Won't have) para identificar qué funcionalidades son estrictamente indispensables para la fecha de entrega.
- Renegociar con los stakeholders un lanzamiento por fases (MVP primero, funcionalidades secundarias en una release posterior), preservando la integridad técnica del sistema sin comprometer la calidad por prisa.

Alternativa 2. Enfoque ágil/híbrido: gestión de flujo y timeboxing con foco en throughput

En lugar de agregar personas, el equipo aplica principios de Kanban/Scrum:

- Limitar el Trabajo en Progreso (WIP limits) para evitar el multitasking y maximizar el throughput real del equipo actual, en vez de asumir que más gente = más velocidad.
- Usar los datos históricos de Cycle Time y Throughput del equipo (métricas de flujo) para generar una proyección probabilística (estilo Monte Carlo) de cuánto se puede entregar realmente en los 2 meses restantes, y comunicar esa fecha con un rango de confianza (ej. "85% de probabilidad de completar X historias") en lugar de prometer el 100% del alcance original.

Esto evita violar los principios de ingeniería de software (no sacrificar calidad ni sobrecargar al equipo) y sustituye la falsa certeza del Gantt fijo por una gestión basada en evidencia empírica.